Exceptions

Joern Ploennigs · AI4SC



> Anything that can go wrong, will go wrong.
>
> — Murphy's Law #1

Exceptions

Das Szenario: KI-Exception analysieren

KI-generierter Code enthält häufig stillschweigende **Annahmen über Eingaben und Rückgabewerte**, die im realen Betrieb verletzt werden. Eine Exception ist kein Zeichen dafür, dass der KI-Code grundsätzlich falsch ist — sie zeigt, welche Annahme der Realität nicht entspricht.

Betrachten wir folgendes Szenario:

```
def hole_messungen(sensor_id):
    sensors = {"sensor_02": [1.2, 3.4, 2.1]}
    return sensors.get(sensor_id)   # gibt None zurück wenn nicht gefunden

def berechne_mittleren_durchfluss(sensor_id):
    messungen = hole_messungen(sensor_id)
    return sum(messungen) / len(messungen)

berechne_mittleren_durchfluss("sensor_01")
```

```
TypeError: 'NoneType' object is not iterable
[31m-------------------------------------------------------------------------[
[39m
[31mTypeError[39m                                 Traceback (most recent
call last)
```

```
[36mCell[39m[36m [39m[32mIn[1][39m[32m, line 9[39m
[32m      6[39m     messungen = hole_messungen(sensor_id)
[32m      7[39m     [38;5;28;01mreturn[39;00m
[38;5;28msum[39m(messungen) / [38;5;28mlen[39m(messungen)
[32m----> [39m[32m9[39m
[43mberechne_mittleren_durchfluss[49m[43m([49m[33;43m"[39;49m[33;43msensor_01[39;49
[49m

[36mCell[39m[36m [39m[32mIn[1][39m[32m, line 7[39m, in
[36mberechne_mittleren_durchfluss[39m[34m(sensor_id)[39m
[32m      5[39m [38;5;28;01mdef[39;00m[38;5;250m
[39m[34mberechne_mittleren_durchfluss[39m(sensor_id):
[32m      6[39m     messungen = hole_messungen(sensor_id)
[32m----> [39m[32m7[39m     [38;5;28;01mreturn[39;00m
[38;5;28;43msum[39;49m[43m([49m[43mmessungen[49m[43m)[49m /
[38;5;28mlen[39m(messungen)

[31mTypeError[39m: 'NoneType' object is not iterable
```

Der vollständige Traceback zeigt:

```
Traceback (most recent call last):
  File "analyse.py", line 9, in <module>
    berechne_mittleren_durchfluss("sensor_01")
  File "analyse.py", line 7, in berechne_mittleren_durchfluss
    return sum(messungen) / len(messungen)
TypeError: 'NoneType' object is not iterable
```

Unterste Zeile: `sum(messungen)` ist abgestürzt, weil `messungen` den Wert `None` hatte. `sum()` erwartet ein iterierbares Objekt, also zum Beispiel eine Liste. Die KI hat angenommen, `hole_messungen()` gibt immer eine Liste zurück — aber `dict.get()` gibt `None` zurück, wenn der Schlüssel nicht existiert.

Tracebacks systematisch lesen

Beim KI-Beispiel lässt sich der Traceback Schritt für Schritt lesen:

1. In der **untersten Zeile** steht der Fehlertyp: `TypeError`.
2. Die **Fehlermeldung** sagt genauer, was schiefging: `NoneType` ist nicht iterierbar.
3. Die Zeile darüber zeigt die **konkrete Absturzstelle**: `sum(messungen)`.
4. Danach liest man **nach oben**, um zu sehen, woher der problematische Wert kam.

So wird aus einem abstrakten Fehlerbericht eine präzise Diagnose: Nicht `sum()` ist falsch, sondern die Vorannahme, dass `messungen` immer eine Liste ist.

Guard Clauses und `try-except`

Guard Clauses prüfen Vorbedingungen **bevor** eine riskante Operation ausgeführt wird. Im Beispiel heißt das: Vor `sum(messungen)` prüfen wir, ob `messungen` überhaupt eine Liste ist und ob sie Werte enthält.

Das ist ein Beispiel für **LBYL** (*Look Before You Leap*): erst prüfen, dann handeln. Python unterstützt aber auch **EAFP** (*Easier to Ask Forgiveness than Permission*): erst versuchen und im Fehlerfall mit `try-except` reagieren.

- **Guard Clauses / LBYL** eignen sich gut, wenn Vorbedingungen fachlich klar sind und die Funktion früh mit einem definierten Ergebnis zurückkehren soll.
- **`try-except` / EAFP** eignet sich gut, wenn eine Operation grundsätzlich erlaubt ist, aber im Einzelfall fehlschlagen kann.

Beide Muster sind nützlich. Entscheidend ist, dass die Annahmen im Code explizit werden und nicht stillschweigend verborgen bleiben.

Fehler mit Exceptions behandeln

Exceptions unterbrechen den normalen Verlauf eines Programmes in einem Fehlerfall. Sie dienen dazu den Fehler im Programm zu kommunizieren und werden genutzt, um im Fehlerfall unkontrolliertes Abstürzen des Programmes zu verhindern.

In KI-generiertem Code signalisieren Exceptions, dass eine **Annahme der KI über Eingaben oder Rückgabewerte verletzt wurde**. Der Traceback zeigt welche Annahme und an welcher Stelle. Das ist wertvolle Diagnoseinformation — kein Grund zur Panik.

Bekannte Exceptions sollten immer abgefangen werden. In Python werden sie durch den `try-except`-Block abgefangen.

```
zaehler = 10
nenner = 0
try:
    ergebnis = zaehler / nenner
    print("Die Division war erfolgreich")
except:
    print("Divisionsfehler")
    ergebnis = None

print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
```

```
Divisionsfehler
Das Ergebnis von 10/0 = None
```

Das `except` ist in dem obigen Fall untypisiert. Damit lassen sich alle Fehler abfangen (auch unbekannte). Das ist aber riskant: Ein bloßes `except:` fängt auch `KeyboardInterrupt` und `SystemExit` ab und kann so das Beenden eines Programms erschweren. Es ist deshalb sinnvoll,

die genaue Exception abzufangen und die Fehlermeldung auszugeben. Der allgemeinste Fehlertyp für normale Fehlerbehandlung ist die `Exception`:

```
zaehler = 10
nenner = 0
ergebnis = None
try:
    ergebnis = zaehler / nenner
except Exception as e:
    print(f"Divisionsfehler mit Fehler vom Typ `{type(e)}` und Meldung `{e}`")

print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
```

```
Divisionsfehler mit Fehler vom Typ `<class 'ZeroDivisionError'>` und Meldung
`division by zero`
Das Ergebnis von 10/0 = None
```

Sind die Exceptions bekannt, sollten sie typisiert behandelt werden, um spezifische Meldungen auszugeben:

```
zaehler = 'keine_zahl'
nenner = 0
ergebnis = None
try:
    ergebnis = zaehler / nenner
except TypeError as e:
    print(f"Zaehler oder Nenner nicht vom Datentyp `int`")
except ZeroDivisionError as e:
    print("Teilung durch 0")

print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
```

```
Zaehler oder Nenner nicht vom Datentyp `int`
Das Ergebnis von keine_zahl/0 = None
```

Der `try-except`-Block unterstützt auch die `else`-Anweisung, welche immer ausgeführt wird, wenn *keine* Exception auftritt. Ferner gibt es die `finally`-Anweisung, welche *immer* ausgeführt wird — also im fehlerfreien Fall und im Fehlerfall. `finally` wird häufig verwendet, um Verbindungen zu Dateien oder Datenbanken zu schließen.

```
zaehler = 2
nenner = 1
ergebnis = None
try:
    ergebnis = zaehler / nenner
```

```
except TypeError as e:
    print(f"Zaehler oder Nenner nicht vom Datentyp `int`")
except ZeroDivisionError as e:
    print("Teilung durch 0")
else:
    print(f"Die Division war erfolgreich: {zaehler}/{nenner} = {ergebnis}")
finally:
    print(f"Ergebnis (immer ausgegeben): {ergebnis}")
```

```
Die Division war erfolgreich: 2/1 = 2.0
Ergebnis (immer ausgegeben): 2.0
```

Semantische Fehler

Semantische Fehler entstehen, wenn ein Programm **syntaktisch korrekt** ist, aber auf ein Problem fachlich oder technisch falsch angewendet wird. Dazu gehören zum Beispiel falsche Datentypen, fehlende Werte oder ungültige Operationen wie eine Division durch Null.

Ein Teil dieser Fehler ist **vorhersehbar**. Dann sollten sie mit Guard Clauses oder `try-except` gezielt behandelt werden. Andere semantische Fehler zeigen sich erst zur Laufzeit und müssen über den Traceback analysiert werden.

Bei KI-generiertem Code ist die Leitfrage besonders nützlich: **Welche Annahme über Daten, Typen oder Rückgabewerte wurde verletzt?**

Exceptions selbst erzeugen

Exceptions können auch selbst mit `raise` erzeugt werden. Das ist sinnvoll, wenn in dem eigenen Code Fehler auftreten können, die woanders behandelt werden müssen. Es ist ratsam, spezifische Fehlertypen zu nutzen, da der Typ des Fehlers viel beim Behandeln hilft.

Als Beispiel definieren wir eine eigene Divisionsfunktion mit expliziten Guard Clauses und klarer Fehlerbenennung:

```
def division(zaehler, nenner):
    if not isinstance(nenner, (int, float)):
        raise TypeError(f"Nenner nicht vom Datentyp `int` oder `float`")
    elif not isinstance(zaehler, (int, float)):
        raise TypeError(f"Zaehler nicht vom Datentyp `int` oder `float`")
    elif nenner == 0:
        print("Warnung Division durch 0")
        return None
    else:
        ergebnis = zaehler / nenner
        print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
        return ergebnis
```

:::::

```
zaehler = "test"
nenner = 0

ergebnis = division(zaehler, nenner)
```

```
TypeError: Zaehler nicht vom Datentyp `int` oder `float`
[31m-------------------------------------------------------------------------[
[39m
[31mTypeError[39m                                  Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[7][39m[32m, line 4[39m
[32m      1[39m zaehler = [33m"[39m[33mtest[39m[33m"[39m
[32m      2[39m nenner = [32m0[39m
[32m----> [39m[32m4[39m ergebnis =
[43mdivision[49m[43m([49m[43mzaehler[49m[43m,[49m[43m
[49m[43mnenner[49m[43m)[49m

[36mCell[39m[36m [39m[32mIn[6][39m[32m, line 5[39m, in
[36mdivision[39m[34m(zaehler, nenner)[39m
[32m      3[39m     [38;5;28;01mraise[39;00m
[38;5;167;01mTypeError[39;00m([33mf[39m[33m"[39m[33mNenner nicht
vom Datentyp `int` oder `float`[39m[33m"[39m)
[32m      4[39m [38;5;28;01melif[39;00m [38;5;129;01mnot[39;00m
[38;5;28misinstance[39m(zaehler, ([38;5;28mint[39m,
[38;5;28mfloat[39m)):
[32m----> [39m[32m5[39m     [38;5;28;01mraise[39;00m
[38;5;167;01mTypeError[39;00m([33mf[39m[33m"[39m[33mZaehler nicht
vom Datentyp `int` oder `float`[39m[33m"[39m)
[32m      6[39m [38;5;28;01melif[39;00m nenner == [32m0[39m:
[32m      7[39m     [38;5;28mprint[39m([33m"[39m[33mWarnung
Division durch 0[39m[33m"[39m)

[31mTypeError[39m: Zaehler nicht vom Datentyp `int` oder `float`
```

```
zaehler = 10
nenner = 2

ergebnis = division(zaehler, nenner)
```

```
Das Ergebnis von 10/2 = 5.0
```

Eigene Exception-Klassen

Für größere Programme reichen eingebaute Exception-Typen nicht immer aus. Dann kann man eine eigene Exception-Klasse definieren, um einen fachlichen Fehler präzise zu benennen:

```
class SensorNotFoundError(Exception):
    pass

def hole_messungen_strikt(sensor_id):
    sensors = {"sensor_02": [1.2, 3.4, 2.1]}
    if sensor_id not in sensors:
        raise SensorNotFoundError(f"Unbekannter Sensor: {sensor_id}")
    return sensors[sensor_id]
```

Eigene Exception-Klassen helfen dabei, fachliche Fehlerfälle klar von allgemeinen Python-Fehlern zu trennen. Aufrufender Code kann dann gezielt `except SensorNotFoundError:` verwenden.

Fehlerbehandlung bei Funktionen im Stack

Exceptions werden im Stack (Aufrufreihenfolge der Funktionen) nach oben weitergegeben, bis sie entweder abgefangen und behandelt werden oder das Programm abstürzt. Der Traceback zeigt diesen Aufrufpfad — das ist die Diagnoseinformation, die uns zeigt, welche Annahme auf welcher Ebene des Stacks verletzt wurde.

Als Beispiel definieren wir eine rekursive Fakultätsfunktion mit einer Guard Clause gegen zu tiefe Rekursion. Die eingebaute Python-Grenze liegt normalerweise viel höher und kann mit `sys.getrecursionlimit()` geprüft werden. Die Grenze `20` hier ist also eine bewusst kleine Lehrgrenze, damit der Effekt schnell sichtbar wird:

```
def factorial_recursiv(x, depth=1):
    if depth > 20:
        raise RecursionError("Recursion zu tief")
    if x > 1:
        return x * factorial_recursiv(x-1, depth+1)
    else:
        return 1
```

```
factorial_recursiv(20)
```

```
2432902008176640000
```

```
factorial_recursiv(21)
```

```
RecursionError: Recursion zu tief
[?][31m----------------------------------------------------------------------------[?]
```

```
[39m
[31mRecursionError[39m                             Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[12][39m[32m, line 1[39m
[32m----> [39m[32m1[39m
[43mfactorial_recursiv[49m[43m([49m[32;43m21[39;49m[43m)[49m

[36mCell[39m[36m [39m[32mIn[10][39m[32m, line 5[39m, in
[36mfactorial_recursiv[39m[34m(x, depth)[39m
[32m      3[39m     [38;5;28;01mraise[39;00m
[38;5;167;01mRecursionError[39;00m([33m"[39m[33mRecursion zu
tief[39m[33m"[39m)
[32m      4[39m [38;5;28;01mif[39;00m x > [32m1[39m:
[32m----> [39m[32m5[39m     [38;5;28;01mreturn[39;00m x *
[43mfactorial_recursiv[49m[43m([49m[43mx[49m[43m-
[49m[32;43m1[39;49m[43m,[49m[43m
[49m[43mdepth[49m[43m+[49m[32;43m1[39;49m[43m)[49m
[32m      6[39m [38;5;28;01melse[39;00m:
[32m      7[39m     [38;5;28;01mreturn[39;00m [32m1[39m

[36mCell[39m[36m [39m[32mIn[10][39m[32m, line 5[39m, in
[36mfactorial_recursiv[39m[34m(x, depth)[39m
[32m      3[39m     [38;5;28;01mraise[39;00m
[38;5;167;01mRecursionError[39;00m([33m"[39m[33mRecursion zu
tief[39m[33m"[39m)
[32m      4[39m [38;5;28;01mif[39;00m x > [32m1[39m:
[32m----> [39m[32m5[39m     [38;5;28;01mreturn[39;00m x *
[43mfactorial_recursiv[49m[43m([49m[43mx[49m[43m-
[49m[32;43m1[39;49m[43m,[49m[43m
[49m[43mdepth[49m[43m+[49m[32;43m1[39;49m[43m)[49m
[32m      6[39m [38;5;28;01melse[39;00m:
[32m      7[39m     [38;5;28;01mreturn[39;00m [32m1[39m

    [31m[... skipping similar frames: factorial_recursiv at line 5 (17
times)][39m

[36mCell[39m[36m [39m[32mIn[10][39m[32m, line 5[39m, in
[36mfactorial_recursiv[39m[34m(x, depth)[39m
[32m      3[39m     [38;5;28;01mraise[39;00m
[38;5;167;01mRecursionError[39;00m([33m"[39m[33mRecursion zu
tief[39m[33m"[39m)
[32m      4[39m [38;5;28;01mif[39;00m x > [32m1[39m:
[32m----> [39m[32m5[39m     [38;5;28;01mreturn[39;00m x *
```

```
[43mfactorial_recursiv[49m[43m([49m[43mx[49m[43m-
[49m[32;43m1[39;49m[43m,[49m[43m
[49m[43mdepth[49m[43m+[49m[32;43m1[39;49m[43m)[49m
[32m      6[39m [38;5;28;01melse[39;00m:
[32m      7[39m     [38;5;28;01mreturn[39;00m [32m1[39m

[36mCell[39m[36m [39m[32mIn[10][39m[32m, line 3[39m, in
[36mfactorial_recursiv[39m[34m(x, depth)[39m
[32m      1[39m [38;5;28;01mdef[39;00m[38;5;250m
[39m[34mfactorial_recursiv[39m(x, depth=[32m1[39m):
[32m      2[39m     [38;5;28;01mif[39;00m depth > [32m20[39m:
[32m----> [39m[32m3[39m         [38;5;28;01mraise[39;00m
[38;5;167;01mRecursionError[39;00m([33m"[39m[33mRecursion zu
tief[39m[33m"[39m)
[32m      4[39m     [38;5;28;01mif[39;00m x > [32m1[39m:
[32m      5[39m         [38;5;28;01mreturn[39;00m x *
factorial_recursiv(x-[32m1[39m, depth+[32m1[39m)

[31mRecursionError[39m: Recursion zu tief
```

Im Traceback oben wird der Stack-Trace sichtbar — die Liste der Funktionsaufrufe die auf dem Stack gesammelt wurde. Um das Abstürzen zu vermeiden, fangen wir den Fehler mit `try-except` ab:

```
import traceback
try:
    factorial_recursiv(21)
except RecursionError as e:
    print(e)
    traceback.print_exc()
```

```
Recursion zu tief
```

```
Traceback (most recent call last):
  File "/var/folders/0c/805v004947lf04d_w7xm50rc0000gn/T/
ipykernel_37938/841887965.py", line 3, in <module>
    factorial_recursiv(21)
  File "/var/folders/0c/805v004947lf04d_w7xm50rc0000gn/T/
ipykernel_37938/43912226.py", line 5, in factorial_recursiv
    return x * factorial_recursiv(x-1, depth+1)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/var/folders/0c/805v004947lf04d_w7xm50rc0000gn/T/
ipykernel_37938/43912226.py", line 5, in factorial_recursiv
```

```
    return x * factorial_recursiv(x-1, depth+1)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/var/folders/0c/805v004947lf04d_w7xm50rc0000gn/T/
ipykernel_37938/43912226.py", line 5, in factorial_recursiv
    return x * factorial_recursiv(x-1, depth+1)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 17 more times]
  File "/var/folders/0c/805v004947lf04d_w7xm50rc0000gn/T/
ipykernel_37938/43912226.py", line 3, in factorial_recursiv
    raise RecursionError("Recursion zu tief")
RecursionError: Recursion zu tief
```

Quiz

Bibliographie